UNIVERSIDAD TECNOLÓGICA DE SAN JUAN DEL RÍO

ACTIVIDAD

CASO PRACTICO

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN E INNOVACIÓN DIGITAL / DESARROLLO DE SOFTWARE MULTIPLATAFORMA / JALPAN

PRESENTA:

ESPINOZA HERNANDEZ CELESTE

PROFESOR DE ASIGNATURA

MARÍA DE MONSERRATH GUERRERO CHAVERO

JALPAN DE SERRA, QRO.

20 DE OCTUBRE DE 2026

INDICE

1. Introducción 3
1.1 Problema que se busca resolver 3
1.2 Objetivo general 3
1.3 Objetivos específicos 4
2. Diseño de la propuesta de solución 4
2.1 Datos principales del sistema 4
2.2 Estructuras de datos básicas 4
2.3 Algoritmos de búsqueda 5
2.4 Algoritmos de ordenamiento 6
2.5 Flujo de préstamo y devolución 7
2.6 Reglas de integridad 7
3. Pruebas realizadas y plan de validación 8
4. Conclusión 9
5. Referencias bibliográficas 10

1. Introducción

La biblioteca escolar requiere administrar títulos, ediciones y ejemplares físicos, además de registrar a los alumnos, los préstamos, las renovaciones, las devoluciones y la condición de cada libro. Cuando estas actividades se realizan mediante registros manuales o archivos dispersos, pueden presentarse duplicidad de identificadores, pérdida de información, dificultad para conocer quién tiene un ejemplar y retrasos al detectar préstamos vencidos.

El caso práctico parte de la necesidad de modernizar la biblioteca mediante un Sistema de Gestión de Biblioteca Escolar (SG-BE), una aplicación web con una base de datos relacional centralizada. El sistema debe distinguir entre la obra o título, la edición bibliográfica, cada ejemplar físico y los movimientos asociados. Esta separación permite que dos copias de un mismo título tengan identificadores, ubicaciones, condiciones y estados de préstamo independientes.

El objetivo de este reporte es proponer el uso de estructuras de datos básicas y algoritmos de búsqueda y ordenamiento para apoyar las operaciones de la biblioteca. Se consideran listas enlazadas para conjuntos dinámicos de registros, colas para solicitudes en espera y pilas para el historial temporal de acciones y la evaluación de expresiones. También se describe cómo localizar y ordenar registros y cómo comprobar el comportamiento mediante casos de prueba.

El alcance del documento es académico: se describe el diseño lógico y las pruebas que deben ejecutarse. Los resultados de las pruebas se presentan como resultados esperados, no como evidencia de una ejecución real del sistema completo.

1.1 Problema que se busca resolver

La biblioteca necesita conocer con precisión qué ejemplares existen, cuáles están disponibles, qué alumno tiene cada libro prestado, cuándo debe devolverlo y en qué condición se recibió. El sistema propuesto reduce la dependencia de registros manuales y proporciona búsquedas, validaciones y reportes consistentes.

1.2 Objetivo general

Diseñar una propuesta de solución que aplique estructuras de datos básicas y algoritmos de búsqueda y ordenamiento para mejorar el control del inventario y la circulación de ejemplares de una biblioteca escolar.

1.3 Objetivos específicos

- Explicar el uso de listas enlazadas, pilas y colas en el contexto bibliotecario.
- Definir algoritmos de búsqueda y ordenamiento apropiados para los registros.
- Analizar la complejidad temporal y espacial de las operaciones principales.
- Establecer pruebas funcionales para préstamos, devoluciones, búsquedas, ordenamientos y operaciones de pila.

2. Diseño de la propuesta de solución

Se propone una aplicación web multiusuario con tres capas lógicas: presentación, aplicación y datos. La capa de presentación contiene formularios y reportes; la capa de aplicación valida reglas de negocio; y la capa de datos mantiene la información persistente en una base de datos relacional. Las estructuras de datos descritas pueden utilizarse en los algoritmos de la capa de aplicación, pero no sustituyen a la base de datos como fuente permanente de información.

2.1 Datos principales del sistema

Entidad	Datos representativos	Finalidad
Obra o título	ID de obra, título, autor	Agrupar la información intelectual de un libro.
Edición	ID de edición, ISBN, editorial, año, idioma	Distinguir las versiones bibliográficas.
Ejemplar	ID único, edición, condición, ubicación, estado	Controlar cada copia física de forma independiente.
Alumno	ID de alumno, nombre, estado	Identificar al prestatario autorizado.
Préstamo	ID, alumno, ejemplar, fecha de salida, vencimiento, devolución	Conservar el ciclo de circulación de un ejemplar.
Movimiento/bitácora	ID, acción, fecha, usuario, observaciones	Mantener trazabilidad de operaciones.

2.2 Estructuras de datos básicas

Lista enlazada

Una lista enlazada está formada por nodos que guardan un dato y una referencia al siguiente nodo. Puede utilizarse para recorrer un conjunto dinámico de préstamos activos o movimientos recientes. Insertar al inicio cuesta O(1), mientras que buscar un registro por un atributo requiere O(n) en el peor caso. Si se necesita conservar los datos después de cerrar la aplicación, los registros deben guardarse también en la base de datos.

Pila dinámica LIFO

La pila funciona bajo el principio LIFO (Last In, First Out): el último elemento insertado es el primero en salir. Se implementa con nodos enlazados y una referencia al tope. Push inserta, Pop retira el elemento superior y Peek consulta el tope sin eliminarlo. Cada una de estas operaciones tiene tiempo O(1).

En la biblioteca, una pila puede servir como estructura temporal para conservar acciones recientes en una función de deshacer, siempre que cada acción guarde información suficiente para revertirse con seguridad. También puede emplearse para evaluar expresiones aritméticas, por ejemplo, en una ampliación que calcule cargos por retraso. Para evaluar expresiones infijas se usan normalmente dos pilas: una de valores y otra de operadores.

Cola FIFO

Una cola sigue el principio FIFO (First In, First Out): el primer elemento que entra es el primero que sale. Puede representar solicitudes de préstamo o reservas pendientes atendidas en orden de llegada. Con una implementación enlazada que mantenga referencias al frente y al final, encolar y desencolar cuestan O(1).

Arreglo

Un arreglo almacena elementos en posiciones indexadas y permite acceder a una posición en O(1). Puede resultar conveniente para conjuntos de códigos ya cargados y ordenados, por ejemplo, para aplicar búsqueda binaria. Insertar o eliminar en medio de un arreglo puede requerir desplazar elementos y costar O(n).

2.3 Algoritmos de búsqueda

Búsqueda lineal

Recorre los registros uno por uno hasta encontrar el elemento solicitado o llegar al final. Es apropiada para una lista enlazada de préstamos o para una colección pequeña que no está ordenada. Su mejor caso es O(1), si el primer elemento coincide; su caso promedio y peor caso son O(n). Usa O(1) de espacio adicional en una versión iterativa.

Búsqueda binaria

Compara la clave buscada con el elemento central de un arreglo ordenado y descarta la mitad que no puede contener el resultado. Su tiempo promedio y peor caso es O(log n), con O(1) de espacio adicional en la versión iterativa. Requiere que los datos estén ordenados y que sea eficiente acceder

a la posición central; por ello, es adecuada para un arreglo ordenado de identificadores, pero no directamente para una lista enlazada.

Para una búsqueda por título o autor en una base de datos, en una implementación real conviene aprovechar consultas e índices de la base de datos. Las complejidades de búsqueda lineal y binaria describen los algoritmos didácticos propuestos, no el comportamiento interno garantizado de un motor SQL.

2.4 Algoritmos de ordenamiento

Ordenamiento burbuja (Bubble Sort)

Compara elementos adyacentes e intercambia los que están en orden incorrecto. Es sencillo para demostrar el ordenamiento de pocos registros por código o título. Con una versión optimizada que detecta si hubo intercambios, el mejor caso es O(n), pero el promedio y peor caso son $O(n^2)$. Utiliza O(1) de espacio adicional.

Ordenamiento por mezcla (Merge Sort)

Divide la colección en partes, ordena cada parte y combina los resultados. Su complejidad temporal en el mejor, promedio y peor caso es O(n log n). Habitualmente requiere O(n) de espacio auxiliar para arreglos. Puede adaptarse a listas enlazadas y es más apropiado que burbuja cuando el volumen de registros crece.

Algoritmo/operación	Tiempo	Espacio auxiliar	Aplicación
Push / Pop / Peek	O(1)	O(1) por operación	Pila dinámica de acciones o evaluación.
Encolar / desencolar	O(1)	O(1) por operación	Cola de solicitudes.
Búsqueda lineal	O(n) peor caso	O(1)	Lista de préstamos o registros sin ordenar.
Búsqueda binaria	O(log n) peor caso	O(1) iterativa	Arreglo ordenado de identificadores.
Bubble Sort optimizado	O(n) mejor; $O(n^2)$ promedio/peor	O(1)	Colección pequeña para fines didácticos.
Merge Sort	O(n log n)	O(n)	Ordenar colecciones de mayor tamaño.
Evaluar expresión con pilas	O(m)	O(m)	Procesar expresión de m caracteres.

n representa el número de registros y m la longitud de una expresión. Las complejidades dependen de la implementación descrita y no incluyen necesariamente el costo de consultar una base de datos o transferir información por la red.

2.5 Flujo de préstamo y devolución

El bibliotecario busca al alumno y el ejemplar por su identificador, título o ISBN.

El sistema comprueba que el alumno esté activo y que el ejemplar esté disponible.

Si las validaciones se cumplen, registra el préstamo con fecha de salida y vencimiento y cambia el estado del ejemplar a prestado.

Al recibir la devolución, registra la fecha, la condición física y las observaciones; calcula el atraso conforme a la política institucional.

El ejemplar pasa a disponible o a revisión, según su condición, y el movimiento queda en la bitácora.

Los reportes muestran inventario, préstamos vigentes, vencidos, daños y pérdidas.

2.6 Reglas de integridad

Cada ejemplar debe tener un identificador único.

No se debe prestar un ejemplar que ya esté prestado, perdido, dado de baja o en revisión.

Un préstamo debe relacionar un alumno y un ejemplar existentes.

La devolución debe conservar fecha, condición y observaciones pertinentes.

Los cambios de estado y movimientos relacionados deben guardarse de forma consistente.

Los permisos deben impedir que usuarios no autorizados alteren inventario o movimientos.

3. Pruebas realizadas y plan de validación

Las siguientes pruebas definen entradas, acciones y resultados esperados para validar la propuesta. Deben ejecutarse en el prototipo o implementación final y registrarse con evidencia. La tabla no afirma que el sistema completo ya se haya ejecutado; funciona como plan de pruebas verificable

ID	Prueba	Entrada/acción	Resultado esperado
P-01	Alta de ejemplar	Registrar un ejemplar con ID nuevo.	Se guarda y aparece como disponible.
P-02	ID duplicado	Registrar otro ejemplar con el mismo ID.	El sistema rechaza el duplicado.
P-03	Préstamo válido	Alumno activo y ejemplar disponible.	Se crea el préstamo y el ejemplar queda prestado.
P-04	Préstamo no permitido	Intentar prestar un ejemplar ya prestado.	La operación se rechaza y no duplica el préstamo activo.
P-05	Devolución	Devolver un ejemplar prestado y registrar condición.	Se registra fecha/condición y se actualiza el estado.
P-06	Préstamo vencido	Fecha de devolución posterior al vencimiento.	El sistema identifica el atraso y aplica la regla configurada.
P-07	Búsqueda lineal	Buscar un ID presente y uno inexistente.	Encuentra el primero e informa que el segundo no existe.
P-08	Búsqueda binaria	Buscar un ID en un arreglo ordenado.	Devuelve la posición correcta o indica que no se encontró.
P-09	Ordenamiento	Ordenar registros por código ascendente.	Los códigos aparecen en orden ascendente.
P-10	Pila LIFO	Push 10, 20, 30; luego Peek y Pop.	Peek devuelve 30; Pop devuelve 30; nuevo tope 20.
P-11	Cola FIFO	Encolar A, B, C y desencolar.	Sale A primero, luego B y después C.
P-12	Expresión	Evaluar 3 + 5 * 2 con pilas.	El resultado es 13, respetando la prioridad.
P-13	División entre cero	Evaluar 18 / (3 - 3).	Se informa el error sin bloquear la aplicación.
P-14	Paréntesis incorrectos	Evaluar (2 + 3 sin cierre.	El evaluador detecta la expresión inválida.

4. Conclusión

El caso de la biblioteca escolar muestra que la selección correcta de estructuras de datos facilita la organización y el procesamiento de la información. Una lista enlazada permite recorrer conjuntos dinámicos; una pila dinámica resulta apropiada para operaciones LIFO y para evaluar expresiones; una cola organiza solicitudes según su orden de llegada; y los arreglos ordenados permiten aplicar búsqueda binaria.

El análisis de complejidad ayuda a comparar soluciones. Las operaciones Push, Pop y Peek de una pila enlazada, así como las operaciones de una cola con referencias al frente y al final, pueden ejecutarse en O(1). La búsqueda lineal requiere O(n) en el peor caso, mientras que la búsqueda binaria alcanza O(log n) si los datos están ordenados y se almacenan en una estructura con acceso eficiente por índice. Bubble Sort es sencillo, pero su costo cuadrático limita su uso en colecciones grandes; Merge Sort ofrece O(n log n) y es más adecuado para volúmenes mayores.

Las estructuras en memoria no sustituyen la persistencia ni las reglas de integridad de una base de datos. Para que la biblioteca sea confiable, la aplicación debe validar los estados, impedir identificadores duplicados, registrar las fechas y condiciones de los ejemplares y conservar una bitácora de movimientos. Las pruebas propuestas permiten comprobar estas reglas y detectar errores antes de utilizar el sistema con datos reales.

En conclusión, combinar estructuras de datos, algoritmos adecuados y una base de datos relacional permite diseñar una solución escalable y verificable para el control bibliotecario. La implementación debe validarse con usuarios de la biblioteca y ajustarse a las políticas reales de préstamo, devolución, renovación y gestión de daños.

5. Referencias bibliográficas

- Oracle. (s. f.). Stack (Java SE 25 & JDK 25). Java Platform, Standard Edition API Specification. https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/Stack.html
- OpenDSA. (s. f.). Stacks. Data Structures and Algorithms. Virginia Tech. https://opendsa.cs.vt.edu/
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.
- Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley Professional.
- Oracle. (s. f.). Java Documentation. https://docs.oracle.com/en/java/
- Sistema de Gestión de Biblioteca Escolar. (2026). Análisis de requisitos, prototipo y plan de desarrollo [Documento de trabajo del caso práctico].